第 4 章 RKNN-Toolkit2 API 介绍

4.1 RKNN 初始化及对象释放

在使用 RKNNToolkit2 的所有 API 接口时，都需要先调用 RKNN()方法初始化 RKNN 对象，当不再使用该对象时，通过调用该对象的 release()方法进行释放。

初始化 RKNN 对象时，可以设置 verbose 和 verbose_file 参数，从而打印详细的日志信息。其中 verbose 参数指定是否要在终端打印详细日志信息；如果设置了 verbose_file 参数，且 verbose 参数值为 True，日志信息还将写到该参数指定的文件中。

举例如下：

```
#将详细的日志信息输出到终端，并写到 mobilenet_build.log 文件中
    rknn=RKNN(verbose=True,verbose_file='./mobilenet_build.log')
    rknn=RKNN(verbose=True)   #将详细的日志信息输出到终端
    …
    rknn.release()
```

4.2 RKNN 模型配置

在构建 RKNN 模型之前，需要先对模型进行通道均值、量化图片 RGB2BGR 转换、量化类型等配置，这些操作需要通过 config 接口进行配置。

API	config
描述	设置要构建的 RKNN 模型属性。
参数	mean_values：输入的均值。参数格式是一个列表，列表中包含一个或多个均值子列表，多输入模型对应多个子列表，每个子列表的长度与该输入的通道数一致，例如[[128,128,128]]，表示一个输入的三个通道的值减去 128。如果 quant_img_RGB2BGR 设置成 True，则优先做 RGB2BGR 转换，再做减均值。
	std_values：输入的归一化值。参数格式是一个列表，列表中包含一个或多个归一化，值子列表，多输入模型对应多个子列表，每个子列表的长度与该输入的通道数一致，例如[[128,128,128]]，表示设置一个输入的三个通道的值减去均值后再除以 128。如果 quant_img_RGB2BGR 设置成 True，则优先做 RGB2BGR 转换，再减均值和除以归一化值。
	quant_img_RGB2BGR：表示在加载量化图像时是否需要先做 RGB2BGR 的操作。默认值为 False，如果有多个输入，则用列表包含起来，如[True,True,False]。该配置一般用在 Caffe 的模型上，Caffe 的模型训练时大多会对先数据集图像进行 RGB2BGR 转换，此时需要将该配置设为 True。 另外，该配置只对量化图像格式为 jpg/jpeg/png/bmp 有效，npy 格式读取时会忽略

	该配置，因此当模型输入为 BGR 时，npy 也需要为 BGR 格式。 注意：该配置仅用于在量化阶段（build 接口）读取量化图像或量化精度分析（accuracy_analysis 接口），并不会记录在最终的 RKNN 模型中，因此如果模型的输入为 BGR，则在调用 Python-API 的 inference 或 C-API 的 run 函数之前，需要保证传入的图像数据也为 BGR 格式。
	quantized_dtype 表示量化类型，目前支持的量化类型有 asymmetric_quantized-8、asymmetric_quantized-16，默认值为 asymmetric_quantized-8。（asymmetric_quantized-16 目前版本暂不支持）
参数	quantized_algorithm 表示计算每一层的量化参数时采用的量化算法，目前支持的量化算法有：norma、mmse 及 kl_divergence，默认采用 normal 量化算法。 normal 量化算法的特点是速度较快，推荐量化数据量一般为 20-100 张左右，更多的数据量下精度未必会有进一步提升。 mmse 量化算法由于采用暴力迭代的方式，速度较慢，但通常会比 normal 具有更高的精度，推荐量化数据量一般为 20-50 张左右，用户也可以根据量化时间长短对量化数据量进行适当增减。 kl_divergence 量化算法耗时比 normal 多一些，但比 mmse 少，在某些场景下（feature 分布不均匀时）可以得到较好的改善效果。
	quantized_method 表示模型量化的方法，目前支持 layer 或者 channel。 layer 表示每层的 weight 只有一套量化参数； channel 表示每层的 weight 的每个通道都有一套量化参数； 通常情况下 channel 会比 layer 精度更高，默认值为 channel。
	float_dtype:浮点的数据类型，目前支持的数据类型有 float16，默认值为 float16。
	optimization_level：模型优化等级。通过修改模型优化等级，可以关掉部分或全部模型转换过程中使用到的优化规则。该参数的默认值为 3，打开所有优化选项。值为 2 或 1 时关闭一部分可能会对部分模型精度产生影响的优化选项，值为 0 时关闭所有优化选项。
	target_platform：指定 RKNN 模型是基于哪个目标芯片平台生成的。目前支持“rk3566”、“rk3568”、“rk3588”、“rv1103”、“rv1106”。该参数的值大小写不敏感。
	custom_stirng：添加自定义字符串信息到 RKNN 模型，可以在 C API 中通过 query 接口查询到该信息。
	remove_weight：去除 conv2d 权重以生成一个 RKNN 的从模型，该从模型可以与带完整权重的 RKNN 模型共享权重以减少内存消耗。默认值为 False。
	compress_weight：压缩模型权重，可以减小 RKNN 模型的大小。默认值为 False。
	single_core_mode（仅对 3588 有效）：是否仅生成单核模型，可以减小 RKNN 模型的大小和内存消耗。默认值为 False。

举例如下：

```
rknn.config(mean_values=[123.675,116.28,103.53],std_values=[58.395,58.395,58.395])
```

4.3 模型加载

RKNN-Toolkit2 目前支持 Caffe、TensorFlow、TensorFlowLite、ONNX、DarkNet、PyTorch 等模型的加载转换，这些模型在加载时需调用对应的接口，以下为这些接口的详细说明。

4.3.1 Caffe 模型加载接口

API	load_caffe
描述	加载 caffe 模型。
参数	model：caffe 模型文件（.prototxt 后缀文件）所在路径。
	blobs：caffe 模型的二进制数据文件（.caffemodel 后缀文件）所在路径。该参数值可以为 None，RKNNToolkit2 将随机生成权重等参数。
	input_name：caffe 模型存在多输入时，可以通过该参数指定输入层名的顺序，形如['input1','input2','input3']，注意名字需要与模型输入名一致；也可不设定，按 caffe 模型文件（.prototxt 后缀文件）自动给定。
返回值	0：导入成功
	-1：导入失败

举例如下：

```
1    #从当前路径加载 mobilenet_v2 模型
2  ret=rknn.load_caffe(model='./mobilenet_v2.prototxt',blobs='./mobilenet_v2.caffemodel')
```

4.3.2 TensorFlow 模型加载接口

API	load_tensorflow
描述	加载 TensorFlow 模型。
参数	tf_pb：TensorFlow 模型文件（.pb 后缀）所在路径。
	inputs：模型的输入节点（操作数名），支持多个输入节点。所有输入节点名放在一个列表中。
	input_size_list：每个输入节点对应的图片的尺寸和通道数。如示例中的 mobilenet-v1 模型，其输入节点对应的输入尺寸是[[1,224,224,3]]。
	outputs：模型的输出节点（操作数名），支持多个输出节点。所有输出节点名放在一个列表中。
返回值	0：导入成功
	-1：导入失败

举例如下：

```
#从当前目录加载 ssd_mobilenet_v1_coco_2017_11_17 模型
    ret=rknn.load_tensorflow(
     tf_pb='./ssd_mobilenet_v1_coco_2017_11_17.pb',inputs=['Preprocessor/sub'],outputs=['concat','concat_1'
       ],input_size_list=[[1,300,300,3]])
```

4.3.3 TensorFlowLite 模型加载接口

API	load_tflite
描述	加载 TensorFlowLite 模型。
参数	model：TensorFlowLite 模型文件（.tflite 后缀）所在路径。
返回值	0：导入成功
	-1：导入失败

举例如下：

```
    #从当前目录加载 mobilenet_v1 模型
    ret=rknn.load_tflite(model='./mobilenet_v1.tflite')
```

4.3.4 ONNX 模型加载

API	load_onnx
描述	加载 ONNX 模型。
参数	model：ONNX 模型文件（.onnx 后缀）所在路径。
	inputs：模型输入节点（操作数名），支持多个输入节点。所有输入节点名放在一个列表中。
	input_size_list：每个输入节点对应的图片的尺寸和通道数。如示例中的 mobilenet-v1 模型，其输入节点对应的输入尺寸是[[1,224,224,3]]。
	outputs：模型的输出节点（操作数名），支持多个输出节点。所有输出节点名放在一个列表中。
返回值	0：导入成功
	-1：导入失败

举例如下：

```
    #从当前目录加载 yolov5 模型
    ret=rknn.load_onnx(model='./yolov.onnx')
```

4.4.5 DarkNet 模型加载接口

API	load_darknet

描述	加载 DarkNet 模型。
参数	model：DarkNet 模型文件（.cfg 后缀）所在路径。
	weight：权重文件（.weights 后缀）所在路径。
返回值	0：导入成功 -1：导入失败

举例如下：

```
#从当前目录加载 yolov3-tiny 模型
ret=rknn.load_darknet(model='./yolov3-tiny.cfg',weight='./yolov3.weights')
```

4.4.6 PyTorch 模型加载接口

API	load_pytorch
描述	加载 PyTorch 模型。 支持量化感知训练（QAT）模型，但需要将 torch 版本更新至 1.9.0 以上。
参数	model：PyTorch 模型文件（.pt 后缀）所在路径，而且需要是 torchscript 格式的模型。必填参数。
	input_size_list：每个输入节点对应的图片的尺寸和通道数。例如[[1,1,224,224],[1,3,224,224]]表示有两个输入，其中一个输入的 shape 是[1,1,224,224]，另外一个输入的 shape 是[1,3,224,224]。必填参数。
返回值	0：导入成功
	-1：导入失败

举例如下：

```
#从当前目录加载 resnet18 模型
ret=rknn.load_pytorch(model='./resnet18.pt',input_size_list=[[1,3,224,224]])
```

4.4 构建 RKNN 模型

API	build
描述	依照加载的模型结构及权重数据，构建对应的 RKNN 模型。
	do_quantization：是否对模型进行量化，值为 True 或 False。

参数	dataset：量化校正数据的数据集。目前支持文本文件格式，用户可以把用于校正的图片（jpg 或 png 格式）或 npy 文件路径放到一个.txt 文件中。文本文件里每一行一条路径信息。如： a.jpg b.jpg 或 a.npy b.npy 如有多个输入，则每个输入对应的文件用空格隔开，如： a.jpg a2.jpg b.jpg b2.jpg 或 a.npy a2.npy b.npy b2.npy 注：量化图片建议选择与预测场景较吻合的图片。 当 npy 文件维度为 4 时，需要将 layout 转为 nhwc。（其余维度不需做此转换）
	rknn_batch_size：模型的输入 Batch 参数调整，默认值为 1。如果大于 1，则可以在一次推理中同时推理多帧输入图像或输入数据，如 MobileNet 模型的原始 input 维度为[1,224,224,3]，output 维度为[1,1001]，当 rknn_batch_size 设为 4 时，input 的维度变为[4,224,224,3]，output 维度变为[4,1001]。注： 1. rknn_batch_size 的调整并不会提高一般模型在 NPU 上的执行性能，但却会显著增加内存消耗以及增加单帧的延迟。 2. rknn_batch_size 的调整可以降低超小模型在 CPU 上的消耗，提高超小模型的平均帧率。（适用于模型太小，CPU 的开销大于 NPU 的开销）。 3. rknn_batch_size 的值建议小于 32，避免内存占用太大而导致推理失败。 4. rknn_batch_size 修改后，模型的 input/output 维度会被修改，使用 inference 推理模型时需要设置相应的 input 的大小，后处理时，也需要对返回的 outputs 进行处理。
返回值	0：构建成功
	-1：构建失败

举例如下：

```
#构建 RKNN 模型，并且做量化
ret=rknn.build(do_quantization=True,dataset='./dataset.txt')
```

4.5 导出 RKNN 模型

通过本工具构建的 RKNN 模型通过该接口可以导出存储为 RKNN 模型文件，用于模型部署。

API	export_rknn
描述	将 RKNN 模型保存到指定文件中（.rknn 后缀）。
参数	export_path：导出模型文件的路径。
返回值	0：导出成功
	-1：导出失败

举例如下：

```
……
#将构建好的 RKNN 模型保存到当前路径的 mobilenet_v1.rknn 文件中
ret=rknn.export_rknn(export_path='./mobilenet_v1.rknn')
```